

REPORTE FINAL DE METODOLOGIA DE LA PROGRAMACION

1. PLANTEAMIENTO DEL PROBLEMA

El proyecto consiste en un programa en Python que simula un sistema de defensa de ciberseguridad. El sistema observa durante 60 ciclos el comportamiento de una red y clasifica el estado operativo como NORMAL, SOSPECHOSO, ALERTA o CRITICO.

El problema a resolver es detectar actividad anomala en un servidor que maneja informacion sensible. Para lograrlo, el programa analiza paquetes de red, intentos de login, cambios de archivos, salida de datos, usuario, IP, CPU, temperatura, cifrado y estado fisico del hardware.

El programa tambien muestra que una defensa digital no solo consiste en detectar ataques. Tambien registra protecciones, bloquea IPs, activa cifrado, calcula consumo y genera graficas y un reporte CSV.

2. OBJETIVO

Desarrollar un simulador modular en Python que use variables, condicionales, ciclos, funciones, listas, diccionarios, operadores logicos, operaciones matematicas, entradas de usuario y salidas en consola, graficas y archivo CSV.

3. ENTRADAS DEL PROGRAMA

El programa usa dos entradas manuales.

3.1 Entrada inicial

Primero muestra la pantalla de inicio y pregunta:

```
python
respuesta_inicio = input("Desea iniciar el programa? (si/no): ").strip().lower()
```

Si el usuario escribe si o s, inicia el ciclo de 60 segundos. Si escribe otra cosa, se imprime:

```
text
Programa finalizado
```

3.2 Entrada final de estabilizacion

Al terminar los 60 ciclos, el programa pregunta:

```
python
respuesta_estabilizar = input("Sistema estabilizado. Quiere desactivar las protecciones? (si/no): ").strip().lower()
```

Si el usuario responde si o s, el estado final se cambia a NORMAL y se apaga la refrigeracion. Si responde cualquier otra cosa, se conserva el estado arrastrado por el sistema, por ejemplo CRITICO.

Esto permite explicar una decision programada:

```
| Respuesta final | Resultado |  
|---|---|  
| si o s | estado final NORMAL |  
| cualquier otra respuesta | conserva el ultimo estado importante |
```

4. ESTRUCTURA MODULAR

```
| Archivo | Funcion |  
|---|---|  
| m00_pdi_slp.py | archivo principal, ciclos, inputs, estado general y reporte final |  
| m01_servidores.py | centrales, usuarios, IPs y umbrales |  
| m02_trafico_normal.py | genera paquetes normales |  
| m03_atacante.py | altera paquetes para simular ataques |  
| m04_logica_matematica.py | calculos, reglas, cifrado, fisica y protecciones |  
| m05_monitor.py | imprime ciclos, paneles y resumen final |  
| m06_graficas.py | genera graficas PNG |  
| m07_reporte_tabla.py | genera el CSV |
```

Esta division ayuda a mantener el codigo organizado por responsabilidades.

5. VARIABLES Y ESTRUCTURAS DE DATOS

El estado principal se guarda en un diccionario:

```
python  
estado_actual = {  
    "estado": "NORMAL",  
    "estado_arrastrado": "NORMAL",  
    "cifrado": "NINGUNO",  
    "refrigeracion": False,  
    "login_bloqueado": False,  
    "ips_bloqueadas": [],  
    "historial": []  
}
```

Tambien se usa un paquete por ciclo:

```
python  
paquete = trafico.generar_trafico_normal(segundo)
```

Ese paquete contiene datos como central, usuario, IP, paquetes, intentos de login, cambios, salida de datos, CPU, temperatura e incidente.

Estructuras utilizadas:

```
| Estructura | Uso |  
|---|---|  
| variables numericas | paquetes, CPU, temperatura, flujo, presion |  
| cadenas | estado, cifrado, usuario, IP, ruta |
```

booleanos	incidente, refrigeracion activa
listas	IPs bloqueadas, historial
diccionarios	paquete, estado actual, metricas

6. PROCESO DEL PROGRAMA

El flujo general es:

text

Inicio -> input inicial -> ciclo 1 a 60 -> calculos -> estado -> protecciones -> reporte cada 5
ciclos -> input final -> resumen final

En cada ciclo:

1. Se genera trafico normal.
2. Se calcula probabilidad de ataque.
3. Si hay ataque, se modifica el paquete.
4. Se calculan tasas, flujo digital, CPU y temperatura.
5. Se evalua el estado del sistema.
6. Se genera ruta cifrada o normal.
7. Se descifra la ruta del servidor como `http://servidor/...`
8. Se actualizan IPs bloqueadas y protecciones.
9. Se calcula ventilacion, refrigeracion, presion y consumo.
10. Se agrega el ciclo al historial.
11. Se imprime el ciclo actual.
12. Cada 5 ciclos se imprime un panel acumulado.

7. CONDICIONALES Y CICLOS

El ciclo principal es:

python

for segundo in range(1, 61):

La probabilidad de ataque cambia por tramos:

text

segundo 1 -> 0.00
segundos 2-20 -> 0.20
segundos 21-40 -> 0.40
segundos 41-55 -> 0.60
segundos 56-60 -> 0.00

Los ultimos 5 ciclos se dejan sin ataques para mostrar una caida clara en la grafica y
representar estabilizacion del sistema.

Tambien se usan condicionales para:

Condicion	Accion

usuario no autorizado	bloquear IP
login critico	cerrar login remoto
flujo critico	activar cifrado reforzado
temperatura alta	activar refrigeracion
respuesta final si	estado final normal

8. SALIDAS DEL PROGRAMA

Las salidas son:

Salida	Descripcion
consola por ciclo	estado, paquetes, CPU, temperatura, ruta y servidor
panel cada 5 ciclos	datos globales, estado del sistema y estado fisico
resumen final	totales, CPU promedio, consumo total y reportes generados
graficas	archivos PNG en pdi_slp_graficas/
CSV	reportes_tabla/pdi_slp_reporte_tabla.csv

El resumen final ya no imprime mensajes extras de guardado. Solo muestra las rutas finales:

text

Reporte grafico : pdi_slp_graficas

Reporte tabular : reportes_tabla/pdi_slp_reporte_tabla.csv

9. RESULTADOS OBTENIDOS

Ultima ejecucion registrada:

Resultado	Valor
ciclos ejecutados	60
incidentes detectados	27
ciclos NORMAL	34
ciclos SOSPECHOSO	2
ciclos ALERTA	15
ciclos CRITICO	9
ultimos 5 ciclos con ataque	0
cifrado final del historial	REFORZADO
CPU promedio	39.3 %
paquetes totales	34513
login totales	134
cambios totales	500
salidas totales	2291 MB
consumo total	16328.57 W

Si el usuario acepta desactivar protecciones al final, el reporte final muestra Estado final: NORMAL. Si no acepta, conserva el estado arrastrado, por ejemplo CRITICO.

10. CONCLUSION

El programa cumple con los elementos de metodologia de la programacion porque tiene entradas, procesamiento, salidas, variables, ciclos, condicionales, funciones, listas, diccionarios y

modularidad.

La solución es clara porque separa la simulación en archivos: uno genera datos, otro simula ataques, otro evalúa reglas, otro imprime resultados, otro grafica y otro genera el CSV. Además, el sistema tiene decisiones programadas: arranque mediante input, detección de ataques, cambio de estado, cifrado, refrigeración y estabilización final.

Como mejora futura, el sistema podría permitir configurar la duración de la simulación, guardar varias ejecuciones con fecha y comparar resultados entre corridas.